



遗传算法

《算法分析与设计》

授课人：罗建超

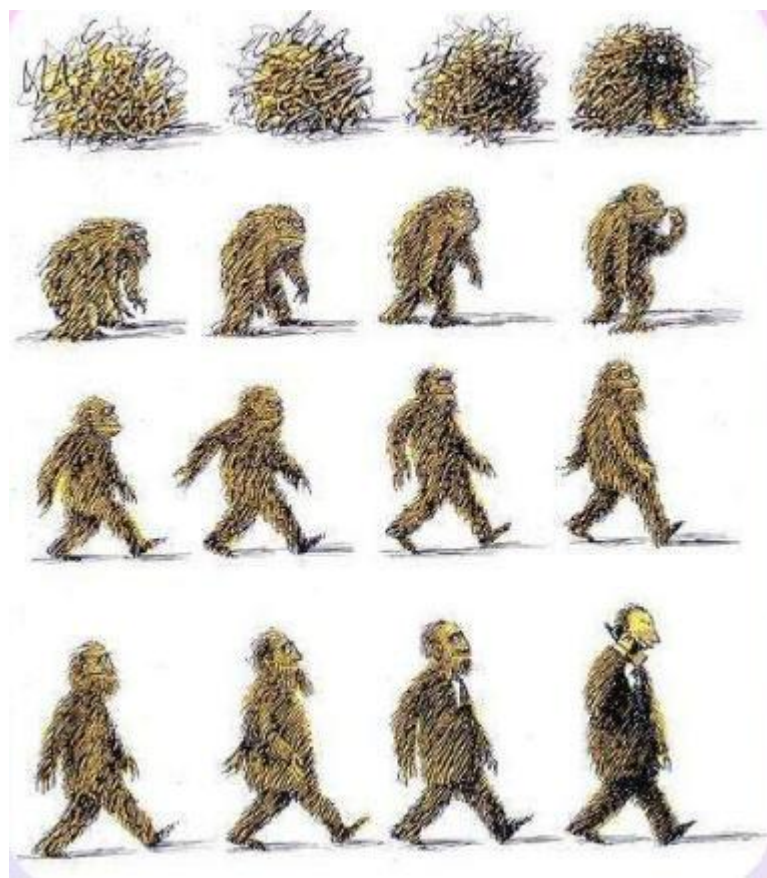
日期：2025年11月14日

遗传算法起源

遗传算法是由美国的J. Holland教授于1975年在他的专著《自然界和人工系统的适应性》中首先提出的，它是一类借鉴生物界**自然选择**和**自然遗传机制**的随机化搜索算法。

遗传算法搜索机制

GA模拟自然选择和自然遗传过程中发生的**繁殖**、**交叉**和**基因突变**现象，在每次迭代中都按**某种指标**从解群中选取较优的个体，利用遗传算子(**选择**、**交叉**和**变异**)对这些个体进行组合，产生新一代的候选解群，重复此过程，直到满足收敛指标为止。



基本遗传算法

基本遗传算法（Simple Genetic Algorithms，简称SGA，又称简单遗传算法或标准遗传算法），是由Goldberg总结出的一种最基本的遗传算法，其遗传进化操作过程简单，容易理解，是其它一些遗传算法的雏形和基础。

基本遗传算法组成

- (1) 编码（产生初始种群）
- (2) 适应度函数
- (3) 遗传算子（选择、交叉、变异）
- (4) 运行参数

GA是通过某种编码机制把对象抽象为由**特定符号按一定顺序排成的串**。正如研究生物遗传是从**染色体**着手，而染色体则是由**基因排成的串**。SGA使用**二进制**或者**数字串**进行编码。

编码：函数优化示例

求下列一元函数的最大值：

$$f(x) = x \cdot \sin(10\pi \cdot x) + 2.0$$

$x \in [0, 3]$ ， 求解结果精确到6位小数。

编码：函数优化示例

由于 $[0, 3]$ 区间长度为3，求解结果精确到6位小数，因此可将自变量定义区间划分为 3×10^6 等份。又因为 $2^{21} < 3 \times 10^6 < 2^{22}$ ，所以本例的二进制编码长度至少需要22位，本例的编码过程实质上是将区间 $[0, 3]$ 内对应的实数值转化为一个二进制串 $(b_{21}b_{20}\dots b_0)$ 。

几个术语

个体，即问题的解

基因型: 1000101110110101000111

基因

解码

编码

表现型: 2.288967

除了用二进制编码还能
怎么样编码？

种群初始化

SGA采用**随机**方法生成若干个**染色体**的集合，该集合称为初始种群。初始种群中个体的**数量**称为**种群规模**。

适应度函数

遗传算法对一个个体（解）的**好坏**用**适应度函数值**来评价，适应度函数设计标准是：**适应度函数值越大，解的质量越好**。适应度函数是遗传算法进化过程的驱动力，也是进行自然选择的唯一标准，它的设计应结合**求解问题本身的要求**而定。

函数优化问题的适应度

求下列一元函数的**最大值**:

$$f(x) = x \cdot \sin(10\pi \cdot x) + 2.0$$

适应度可以表示为染色体 ($b_{21}b_{20}\dots b_0$) 解码后对应的值 x 的**函数值** $f(x)$.

函数优化问题的适应度

求下列一元函数的**最小值**：

$$f(x) = x \cdot \sin(10\pi \cdot x) + 2.0$$

适应度可以表示为染色体（ $b_{21}b_{20}\dots b_0$ ）解码后对应的值 x 的**函数值** $1/f(x)$ 或**MAX- $f(x)$** 。

选择算子

遗传算法使用选择运算来实现对群体中的个体进行**优胜劣汰**操作：**适应度高**的个体被遗传到下一代群体中的**概率大**；**适应度低**的个体，被遗传到下一代群体中的**概率小**。选择操作的任务就是按某种方法从父代群体中选取一些个体，遗传到下一代群体。SGA中选择算子采用**轮盘赌**选择方法。

轮盘赌选择法

轮盘赌选择又称**比例选择算子**，它的基本思想是：各个个体被**选中的概率**与其**适应度值**大小**成正比**。设群体大小为 n ，个体 i 的适应度为 F_i ，则个体 i 被选中遗传到下一代群体的概率为：

$$P_i = F_i / \sum_{i=1}^n F_i$$

轮盘赌选择法原理

个体**适应度**越大，占
总的适应度的**占比**越
大，个体被选中的**概**
率越高。



轮盘赌选择法的实现步骤

- (1) 计算群体中**所有个体的适应度**函数值；
- (2) 利用**比例选择算子**的公式，计算每个个体被选中遗传到下一代群体的**概率**；
- (3) 采用**模拟赌盘操作**（即生成0到1之间的随机数与每个个体遗传到下一代群体的概率进行匹配）来确定各个个体是否遗传到下一代群体中。

其它选择方式探讨

除了用轮盘赌还能怎么
选择？

交叉算子

所谓交叉运算，是指对两个相互配对的染色体依据交叉概率 P_c 按某种方式相互交换其部分基因，从而形成两个新的个体。交叉运算是遗传算法区别于其他进化算法的重要特征，它在遗传算法中起关键作用，是产生新个体的主要方法。SGA中交叉算子采用单点交叉算子。

单点交叉运算

交叉点

交叉前（父代）：

00000|01110000000010000

11100|00000111111000101

交叉后（子代）：

00000|00000111111000101

11100|01110000000010000

还能怎么交叉运算？

基本位变异算子

基本位变异算子是指对个体编码串**随机指定的某一位或某几位**基因作变异运算。对于基本遗传算法中用二进制编码符号串所表示的个体，若需要进行变异操作的**某一基因**的原有基因值为**0**，则变异操作将其变为**1**；反之，若原有基因值为**1**，则变异操作将其变为**0**。

基本位变异算子的执行过程

变异点

变异前：

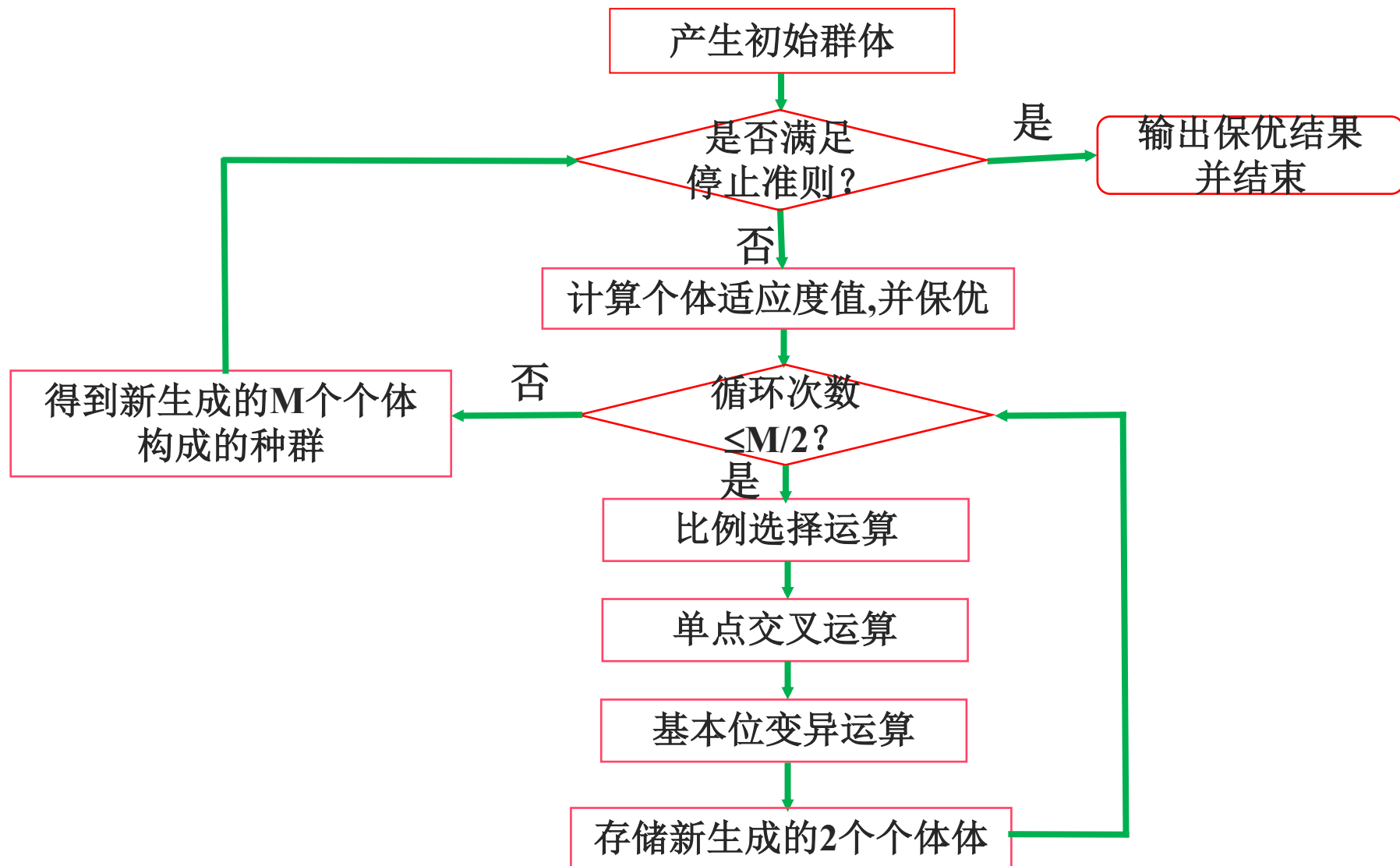
0000011100000000010000

变异后：

0000011100010000010000

还能怎么变异运算？

遗传算法执行流程



遗传算法参数

(1) M : 种群规模

(2) P_c : 交叉概率

(3) P_m : 变异概率

(4) T : 终止进化代数 (停止规则)

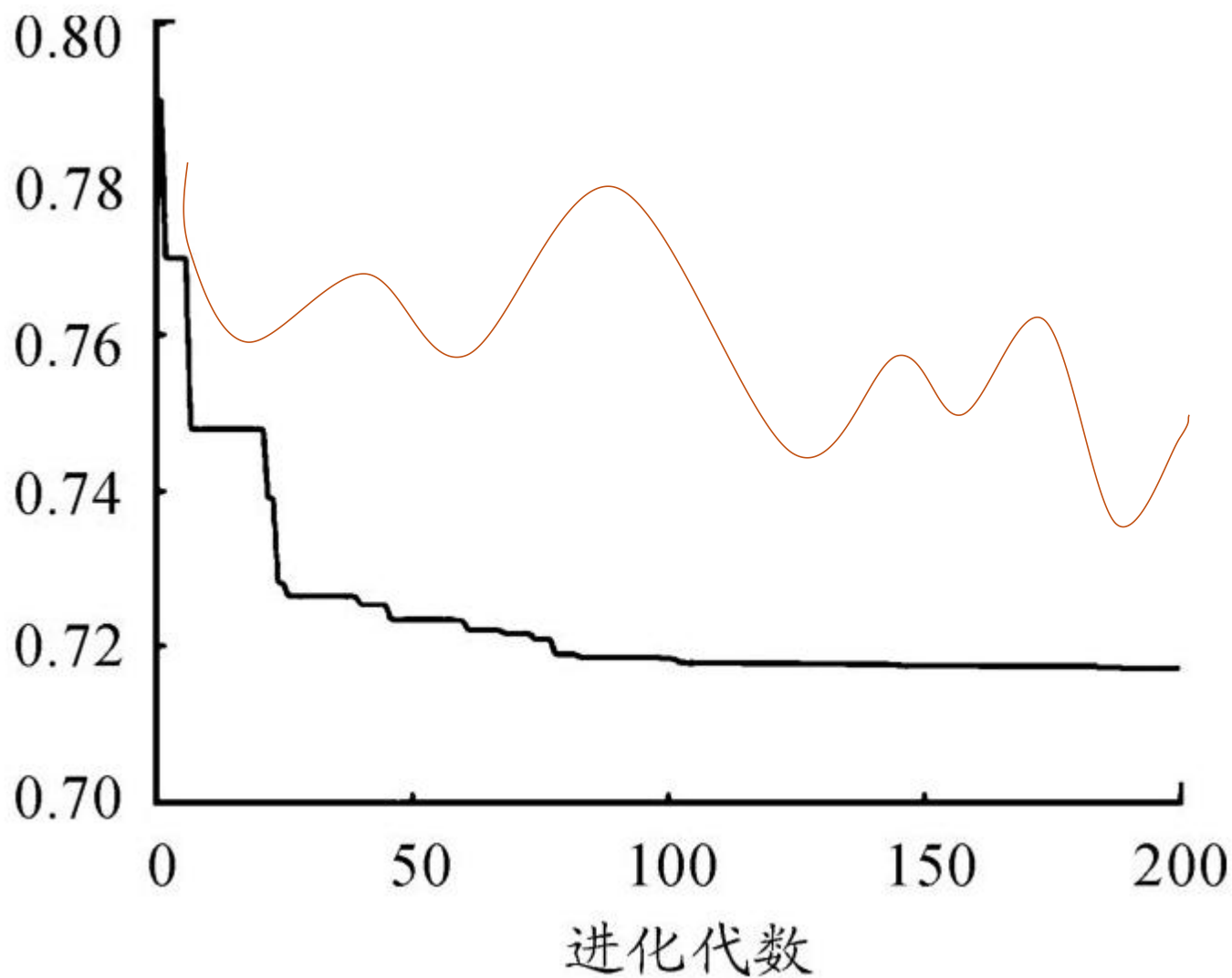
遗传算法的特点

- (1) 群体搜索，易于并行化处理；
- (2) 自组织、自适应和自学习特征；
- (3) 不要求导和其它辅助知识，只需要知道适应度函数；
- (4) 强调概率转换规则，而非确定的转换规则。

遗传算法收敛性分析

遗传算法要实现全局收敛，首先要求任意初始种群经**有限步**都能到达全局最优解，其次算法**必须有**保优操作来防止最优解的遗失。与算法收敛性有关的**因素**主要包括**种群规模**、**选择操作**、**交叉概率**和**变异概率**。

遗传算法收敛性分析



种群规模对收敛性影响

通常，**种群太小**则不能提供足够的采样点，以致算法**性能很差**；**种群太大**，尽管可以增加优化信息，阻止早熟收敛的发生，但无疑会增加计算量，**造成收敛时间太长**，表现为收敛速度缓慢。

选择操作对收敛性影响

选择操作使高适应度个体能够以更大的概率生存，从而提高了遗传算法的全局收敛性。

如果在算法中采用最优保存策略，即将父代群体中最佳个体保留下来，不参加交叉和变异操作，使之直接进入下一代，最终可使遗传算法以概率1收敛于全局最优解。

交叉概率对收敛性的影响

交叉操作作用于个体对，产生新的个体，**实质上是在解空间中进行有效搜索**。交叉概率**太大**时，种群中个体更新很快，会造成**高适应度值的个体很快被破坏掉**；概率**太小时**，交叉操作很少进行，从而使搜索停滞不前，造成算法的**不收敛**。

变异概率对收敛性的影响

变异操作是对种群模式的**扰动**，有利于增加种群的多样性。但是，变异**概率太小**则很难产生新模式，**变异概率太大**则会使遗传算法成为**随机搜索算法**。

控制参数的改进

Schaffer建议的最优参数范围是：

$$\mathbf{M = 20-100,}$$

$$\mathbf{T = 100-500,}$$

$$\mathbf{P_c = 0.4-0.9,}$$

$$\mathbf{P_m = 0.001-0.01.}$$

控制参数的改进

Srinivas等人提出**自适应遗传算法**，即 P_c 和 P_m 能够随适应度自动改变，当种群的各个**个体适应度趋于一致或趋于局部最优**时，使二者**增加**，而当种群适应度**比较分散**时，使二者**减小**，同时对适应值**高于群体平均适应值的个体**，采用**较低的 P_c 和 P_m** ，使性能优良的个体进入下一代，而**低于平均适应值的个体**，采用**较高的 P_c 和 P_m** ，使性能较差的个体被淘汰。

对执行策略的改进

混合遗传算法（模拟退火与遗传算法）

免疫遗传算法（免疫算法与遗传算法）

小生境遗传算法（按小生境方式分类）

单亲遗传算法（遗传操作只有单个个体完成）

并行遗传算法（适应度、遗传操作、分组并行）

遗传算法的应用

遗传算法提供了一种求解复杂系统优化问题的通用框架，它不依赖于问题的具体领域，对问题的种类有很强的鲁棒性，所以广泛应用于很多学科。下面是遗传算法的一些主要应用领域：

(1) 函数优化

函数优化是遗传算法的经典应用领域，也是对遗传算法进行性能评价的常用算例。对于一些非线性、多模型、多目标的函数优化问题，用其他优化方法较难求解，用遗传算法可以方便地得到较好的结果。

(2) 组合优化

随着问题规模的增大，组合优化问题的搜索空间也急剧扩大，有时在目前的计算机上用枚举法很难或甚至不可能求出其精确最优解。对这类复杂问题，人们已意识到应把主要精力放在寻求其**满意解**上，而遗传算法是寻求这种满意解的最佳工具之一。实践证明，遗传算法对于组合优化中的NP完全问题非常有效。

例如，遗传算法已经在求解**旅行商问题**、**装箱问题**、**图形划分问题**等方面得到成功的应用。

遗传算法的应用

(3) 生产调度问题

生产调度问题在很多情况下所建立起来的数学模型难以精确求解，即使经过一些简化之后可以进行求解，也会因简化得太多而使得求解结果与实际相差甚远。而目前在现实生产中也主要是靠一些经验来进行调度。

现在遗传算法已成为解决复杂调度问题的有效工具，在单件生产车间调度、流水线生产车间调度、生产规划、任务分配等方面遗传算法都得到了有效的应用。

(4) 自动控制

在自动控制领域中很多与优化相关的问题需要求解，遗传算法已在其中得到了初步的应用，并显示出了良好的效果。

例如用遗传算法进行航空控制系统的优化、使用遗传算法设计空间交会控制器、基于遗传算法的模糊控制器的优化设计、基于遗传算法的参数辨识、基于遗传算法的模糊控制规则的学习、利用遗传算法进行人工神经网络的结构优化设计和权值学习等，都显示出了遗传算法在这些领域中应用的可能性。

遗传算法的应用

(5) 机器人学

机器人是一类复杂的难以精确建模的人工系统，而遗传算法的起源就来自于对人工自适应系统的研究，所以机器人学理所当然地成为遗传算法的一个重要应用领域。

例如，遗传算法已经在移动机器人**路径规划**、关节机器人运动轨迹规划、机器人逆运动学求解、细胞机器人的**结构优化和行为协调**等方面得到研究和应用。

(6) 图像处理

图像处理是计算机视觉中的一个重要研究领域。在图像处理过程中，如扫描、**特征提取、图像分割**等不可避免地会存在一些误差，这些误差会影响图像处理的效果。如何使这些误差最小是使计算机视觉达到实用化的重要要求。

遗传算法在这些图像处理中的优化计算方面找到了用武之地，日前已在模式识别、图像恢复、图像边缘特征提取等方面得到了应用。

(7) 人工生命

人工生命是用计算机、机械等人工媒体模拟或构造出的具有自然生物系统特有行为的人造系统。自组织能力和自学习能力是人工生命的两大主要特征。人工生命与遗传算法有着密切的关系，基于遗传算法的进化模型是研究人工生命现象的重要基础理论。

旅行商问题

旅行商问题（TSP）是组合优化中研究最多的问题之一。该问题的叙述如下：一个商人欲从自己所在的城市出发，到若干个城市推销商品，然后回到其所在的城市。如何选择一条周游路线使得商人经过每个城市一次且仅一次后回到起点且使他所走过的路径最短？

编码

假设一共有 $n + 1$ 个城市，则可以对每个城市编码为 $1, 2, \dots, n + 1$ 。问题的解即为一个**2到 $n + 1$** 的排列。若 $n = 9$ ，则一个**染色体**可以表示为：

$$\Delta = (2, 8, 5, 3, 4, 6, 10, 7, 9)$$

其含义为：1-2-8-5-3-4-6-10-7-9-1

适应度函数

令 $f(\Delta)$ 表示从城市1依次经过 Δ 中的城市需要走的总距离。

则适应度函数可以表示为：

$$F(\Delta) = 1/f(\Delta)$$

交叉操作

交叉点

交叉前:

2, 8, 5, 3, | 4, 6, 7, 9, 10

4, 6, 7, 2, | 8, 5, 3, 10, 9

交叉后:

2, 8, 5, 3, | 8, 5, 3, 10, 9

4, 6, 7, 2, | 4, 6, 7, 9, 10

修复操作

待修复个体:

$$\Pi = \{2, 8, 5, 3, 8, 5, 3, 10, 9\}$$

扫描 Π ，统计基因出现的次数，将非第一次出现的基因替换为0:

$$\Pi_1 = \{2, 8, 5, 3, 0, 0, 0, 10, 9\}, \text{Count} = \{1, 2, 0, 2, 0, 0, 2, 1, 1\}$$

将Count为0的基因取出来打乱顺序后，依次替换 Π_1 中为0的基因，假设 $\{4, 6, 7\}$ 打乱后为 $\{7, 4, 6\}$ ，则修复后的染色体为:

$$\Pi_2 = \{2, 8, 5, 3, 7, 6, 4, 10, 9\}$$

变异操作

变异前:

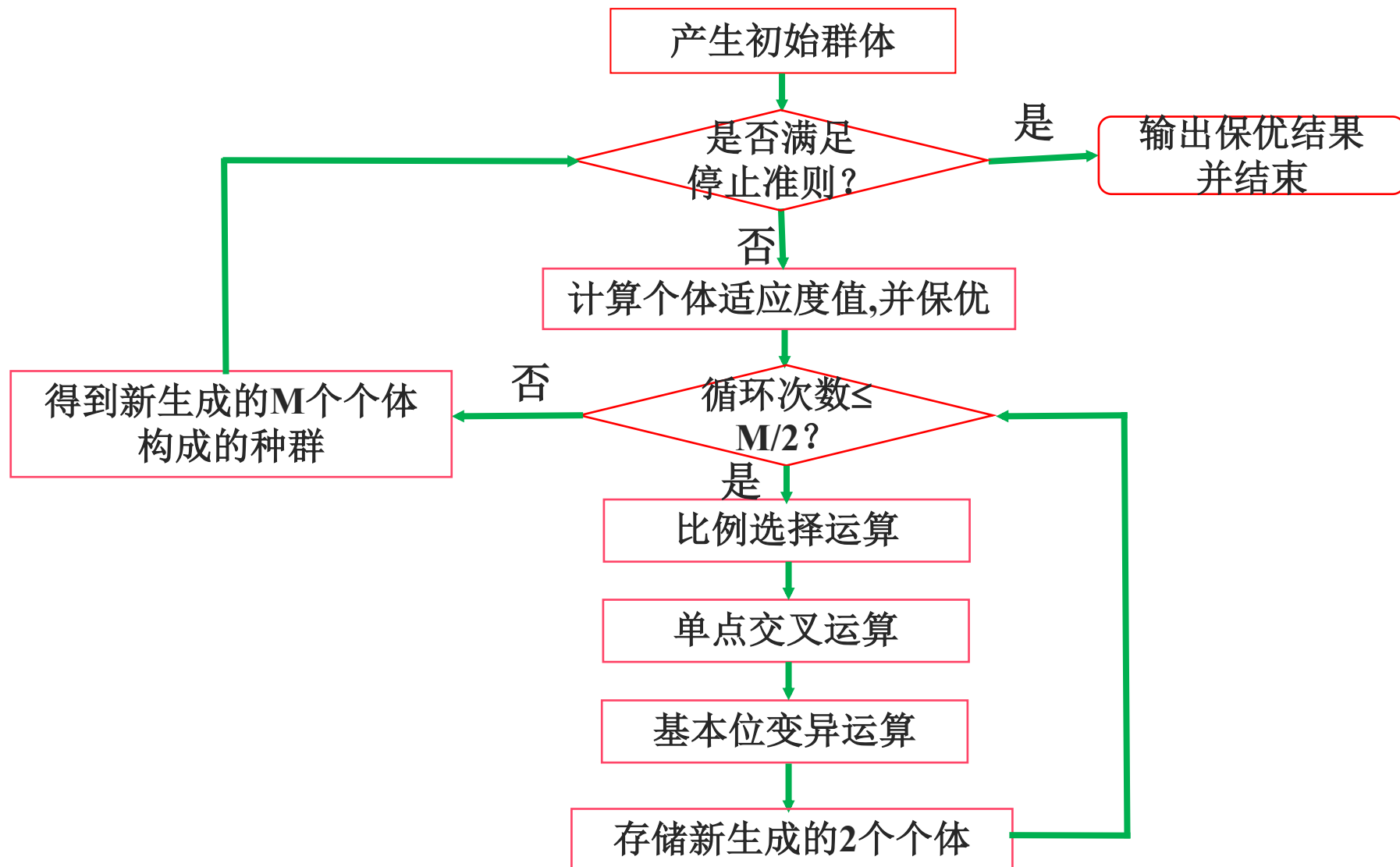
2, 8, 5, 3, |4, 6, 7, 9, 10

变异后:

2, 8, 5, 3, |7, 6, 7, 9, 10

变异点

遗传算法执行流程



遗传算法用于神经网络的权值优

- (1) 采用某种**编码**方案对每个权值进行编码，随机产生一组权值编码；
- (2) 计算神经网络的**误差函数**，确定其适应度的函数值，误差值越大，**适应度值**越小；
- (3) 选择若干**适应度值**大的个体直接遗传给下一代，其余按适应值确定的概率遗传；
- (4) 利用交叉、变异等操作处理当前种群，产生下一代种群；
- (5) 重复(2)～(3)，直到取得满意解。

展望

